

Adding 3-D UAV Mobility and Flight Energy to the Energy-Aware UAV-Fog Simulation

A visual guide to the design, grounded in FU-Serve and Con-Fog, using the Zeng19 power model

Previously the UAVs were placed at random positions and never moved — they only ever hovered. This guide explains the changes that make the swarm **fly**: where UAVs go, how they spread out without colliding, how their flight energy is charged against the same Zeng19 power model already in the code, and how all of this flows through the routing pipeline. Every design choice is kept faithful to the two reference papers so it is easy to defend in review and easy to read in the simulation results.

Design choice	What it means	Grounded in
Horizontal motion only	UAVs fly at a fixed altitude; Zeng19 governs the power	Zeng19 Eq. (3)
Fixed altitude z	z appears in 3-D distance but never changes (no climb)	FU-Serve Eq. (10)
Event-driven (default)	UAVs track their assigned IoT demand hotspot	FU-Serve / Con-Fog
Random (ablation)	Random-waypoint motion; demand-agnostic baseline	Con-Fog §V-B
Static (baseline)	No motion; reproduces the original sim byte-for-byte	original code
Ring + altitude layers	Co-assigned UAVs separated in x-y and in altitude	no-collision by design

1. The energy model: one curve for hover and flight

Flight energy is not a new formula bolted on. The code already contains the Zeng19 rotary-wing power curve $P(V)$; we simply use it. A hovering UAV is just the $V = 0$ point of the same curve, so there is no discontinuity and nothing to reconcile. Each control tick, a UAV is charged $P(V) \times 0.1$ s of propulsion energy at its current speed.

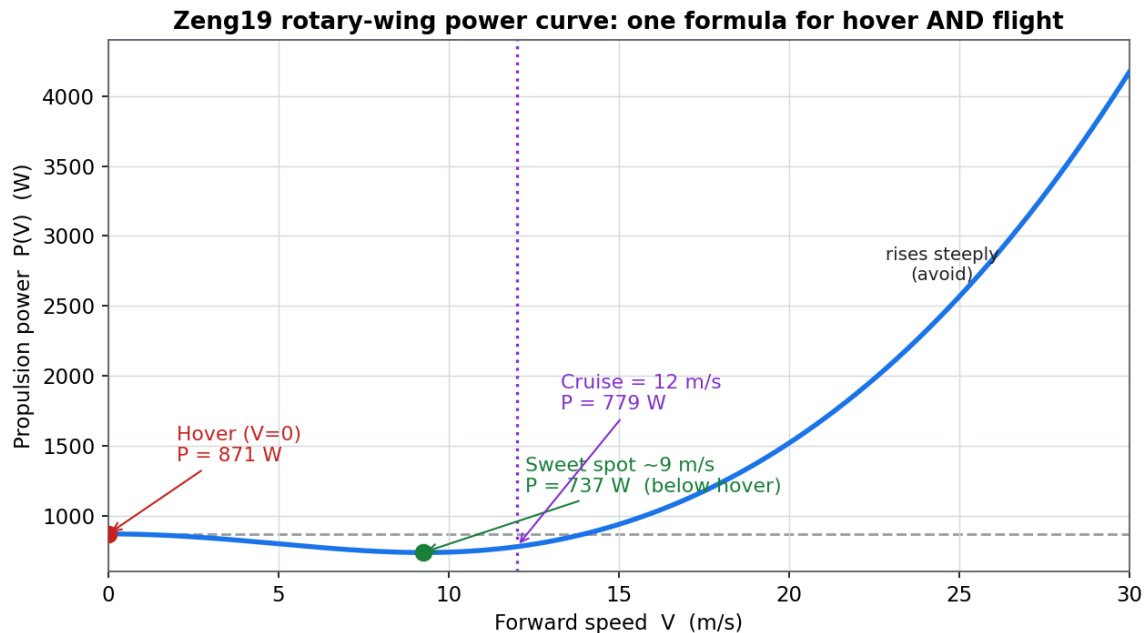


Figure 1. Propulsion power vs forward speed. Hovering costs 871 W. Cruising near 9–12 m/s actually costs *less* than hovering (the induced-power dip), and power rises steeply only above ~20 m/s. We cruise at 12 m/s to keep flight cheap.

Why this matters in the results: because cruising at 12 m/s is cheaper than hovering, a moving swarm can spend *less* propulsion energy than a stationary one. That counter-intuitive-but-correct effect shows up directly in Figure 6.

2. Where mobility enters the control loop

The 100 ms control tick is unchanged in shape. Mobility adds two steps at the top: move the UAVs, then recompute the geometry so the new positions feed every downstream calculation. Routing, reward and reporting are exactly as before.

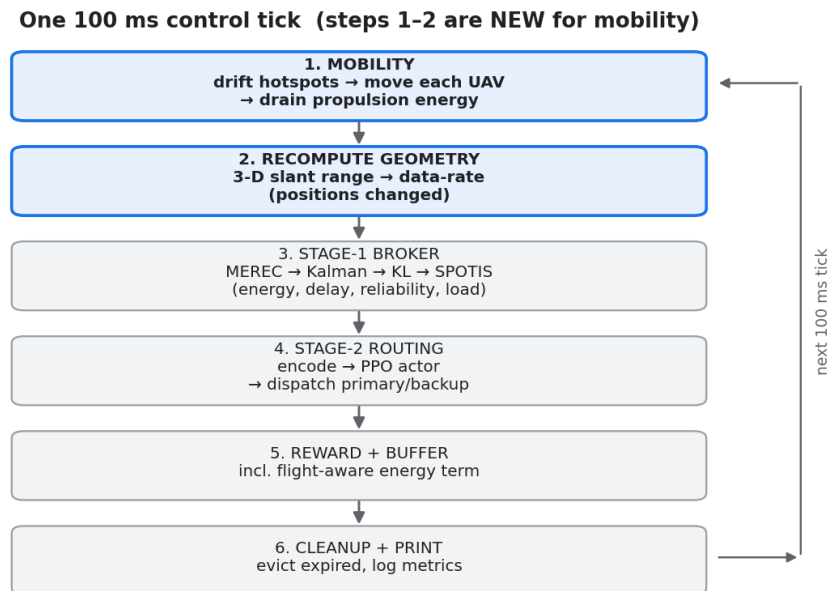


Figure 2. One control tick. Steps 1–2 (blue) are new: drift demand, move each UAV and drain propulsion energy, then refresh distances/data-rates. The rest of the pipeline is untouched.

3. How UAVs share a hotspot without colliding

Five UAVs are assigned to each IoT hotspot. They do **not** stack on the centroid. Each takes a distinct *slot* on a 250 m standoff ring (so they blanket the cluster) and a distinct *altitude layer* in the 80–150 m band. Even if two stations overlapped horizontally, they are separated in altitude — so collisions cannot occur and no avoidance controller is needed (matching how FU-Serve and Con-Fog treat UAVs as repositioning points).

Each hotspot's UAVs spread on a ring AND across altitude layers

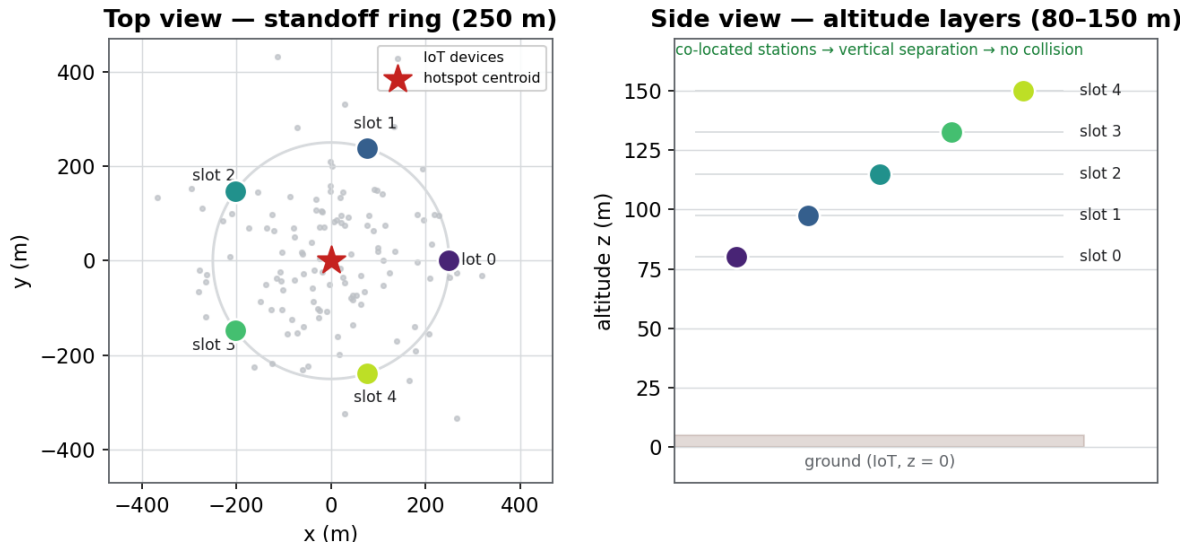


Figure 3. Left: the five UAVs of one hotspot sit on a ring around the demand cluster. Right: the same five are layered across altitude, giving vertical separation.

4. How movement propagates through the pipeline

Moving a UAV changes its 3-D slant range to the ground devices (faithful to FU-Serve Eq. 10). That single change ripples outward: data-rate, delay, reliability and communication energy all shift, which is what the MCDM broker and the PPO router then act on. Separately, the UAV's speed sets its propulsion power and drains the battery.

How movement propagates through the whole pipeline

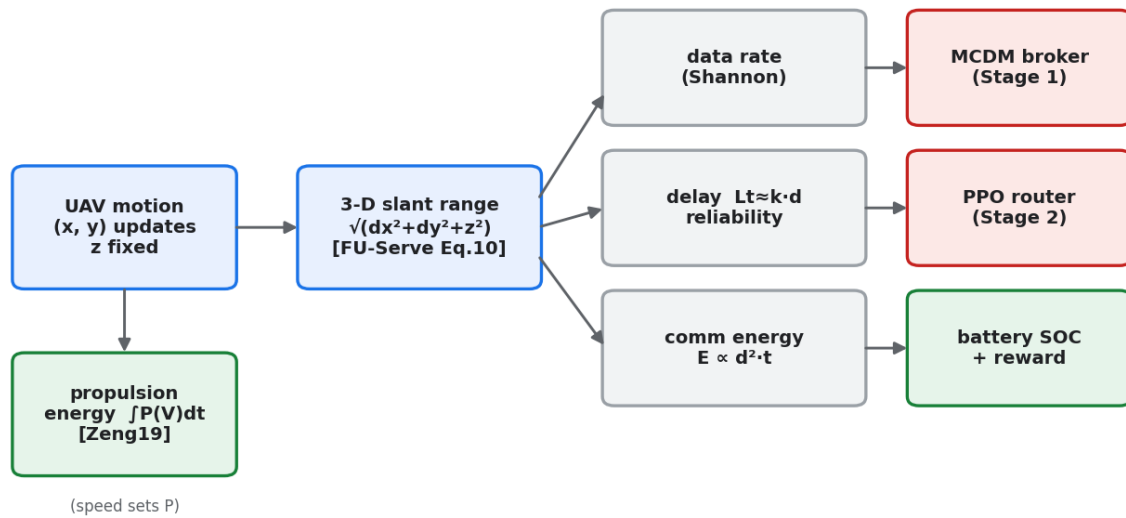


Figure 4. The chain of consequences from a position update. Blue = new mobility blocks; grey = link/QoS quantities; red = the two decision stages; green = energy.

5. What the two movement modes look like

event_driven (the default) sends each UAV to its assigned demand hotspot and keeps it there as the hotspot slowly drifts — demand-following. **random** is the ablation baseline: UAVs chase random waypoints, ignoring demand. Comparing the two isolates the benefit of demand-aware repositioning.

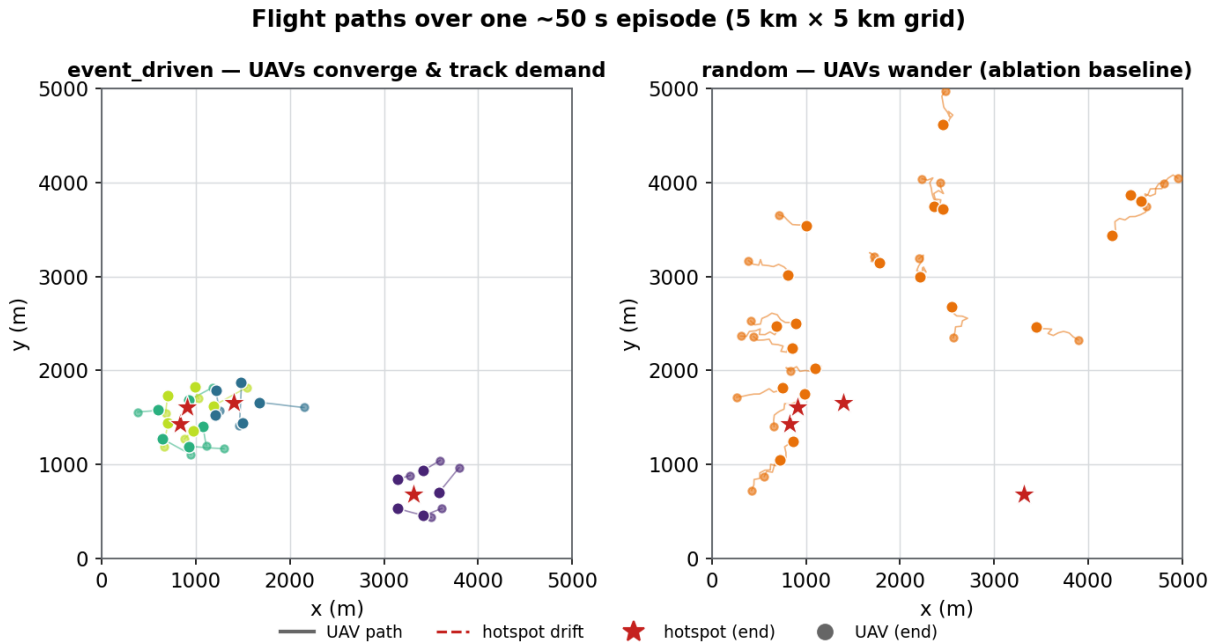


Figure 5. Flight paths over one ~50 s episode. Left: event_driven UAVs (coloured by hotspot) cluster on the drifting demand centres (red stars). Right: random UAVs wander, unrelated to demand.

static is the third mode: UAVs never move, reproducing the original fixed-position simulation exactly — the clean before/after reference.

6. Reading the results: proximity to demand

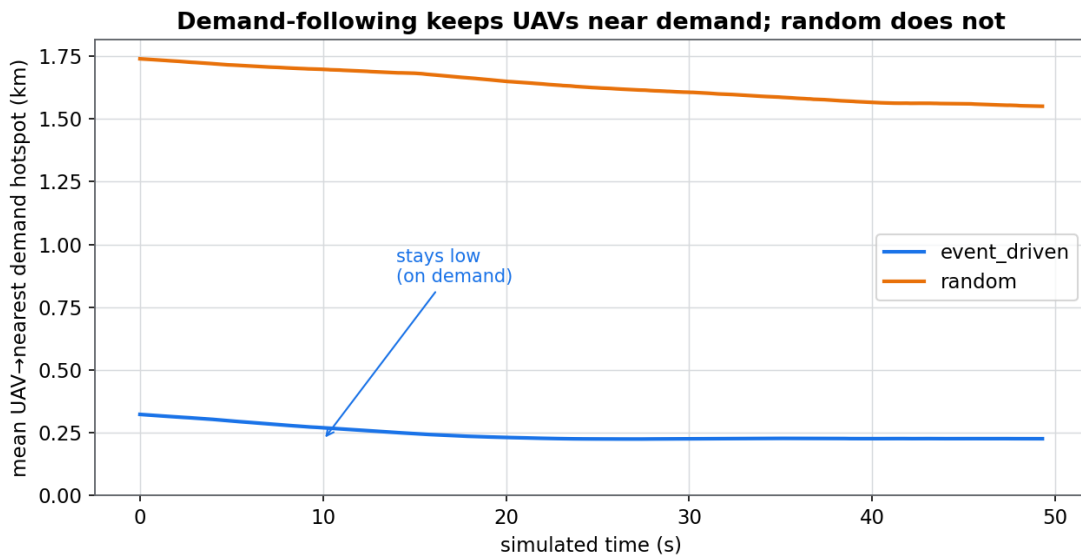


Figure 6. Mean distance from each UAV to the nearest demand hotspot. event_driven settles near the ring radius (~0.23 km) and stays there; random sits ~1.5 km away throughout. Closer to demand means shorter links, lower delay and lower comms energy.

7. Reading the results: energy due to movement

This is the headline energy figure. Propulsion power for `event_driven` starts at the cheap cruise level (~788 W) and climbs toward the hover cost (871 W) as UAVs arrive on station; `random` stays at the ~779 W cruise level because it never stops moving. The result: total propulsion energy is *highest for static hover* (860 kJ) and lowest for continuous cruise (770 kJ) — a direct, visible consequence of the Zeng19 power curve in Figure 1.

Energy expenditure due to movement (Zeng19 curve, integrated per tick)

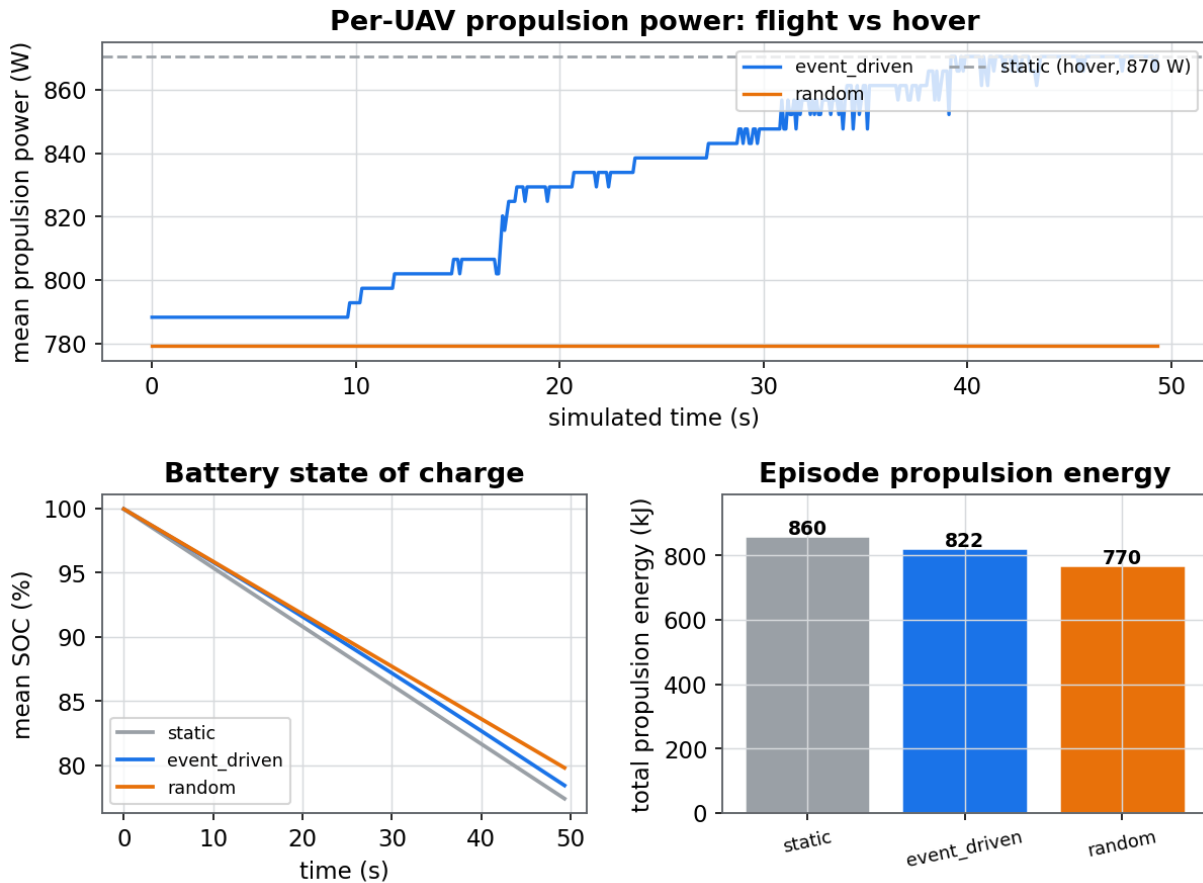


Figure 7. Top: per-UAV propulsion power over the episode. Bottom-left: battery state of charge — static (pure hover) drains fastest. Bottom-right: total propulsion energy per mode.

Talking point for reviewers / colleagues: enabling realistic movement did not just add a cost — it exposed a genuine energy trade-off the routing agent can exploit, and the *static* mode still reproduces the original numbers exactly, so the comparison is clean.

Modes are selected with `MOVE_MODE` (`event_driven` / `random` / `static`) in `config.py` or the `UAV_MOVE_MODE` environment variable. Figures 5–7 are from one untrained evaluation episode per mode (seed 42); the mechanism, not policy quality, is what they illustrate.